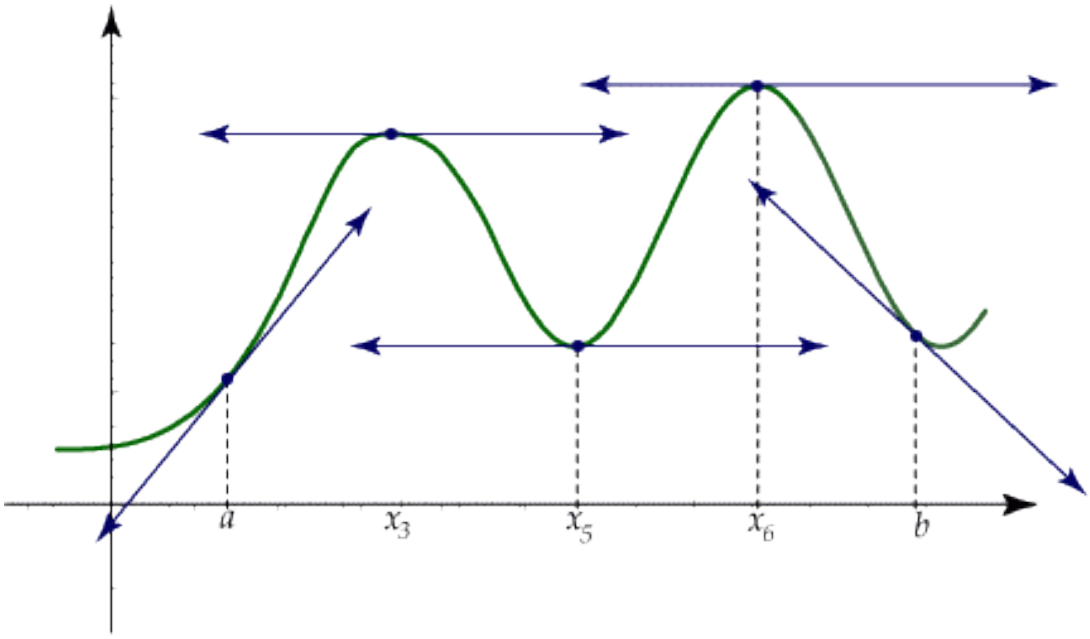


Actividad 1: Optimización de funciones unidimensionales

MATLAB

Sergio Casado López | Optimización | 01/05/2026

Indice

Método de búsqueda dicotómica 2
Metodo de interpolacion cuadratica 3
Método de Newton 4
Método de las rectas inexactas 5
Script principal de ejecución 6
Resultados de los métodos de búsqueda 8
- Método de búsqueda dicotómica 9
- Método de interpolación cuadrática 9
- Método de Newton 10
- Método de las rectas inexactas 11

Análisis de los resultados 12
Enlace al repositorio de GitHub 13

Método de búsqueda dicotómica

El método de búsqueda dicotómica es una técnica iterativa utilizada para encontrar el mínimo de una función en un intervalo donde se sabe que es unimodal, es decir, que presenta un único mínimo. El procedimiento consiste en reducir progresivamente el intervalo de búsqueda evaluando la función en dos puntos cercanos al punto medio del intervalo actual.

En cada iteración, se calcula el punto medio del intervalo y se evalúa la función en dos puntos ligeramente desplazados a ambos lados de dicho punto. Comparando los valores obtenidos, se determina en qué subintervalo se encuentra el mínimo y se descarta la otra mitad del intervalo. Este proceso se repite hasta que la longitud del intervalo es suficientemente pequeña, lo que permite aproximar el valor del mínimo con la precisión deseada.

```
function [xmin, fmin, iter, tiempo] = budi(f, a, b, tol, delta)

% ---------------- INICIALIZACIÓN ----------------
tic;
iter = 0;
maxIter = 10000;   % Por seguridad, para que no esté realizando
iteraciones infinitas

% ---------------- BUCLE PRINCIPAL ----------------
while (b - a) > tol && iter < maxIter

    m = (a + b)/2;

    x1 = m - delta;
    x2 = m + delta;

    % Evaluaciones de la función en los extremos del intervalo
    f1 = f(x1);
    f2 = f(x2);

    % Modificación del intervalo
    if f1 < f2
        b = x2;
    else
        a = x1;
    end

    iter = iter + 1;
end

% ---------------- RESULTADO ----------------
xmin = (a + b)/2;
fmin = f(xmin);
tiempo = toc;

end
```

Metodo de interpolacion cuadratica

El método de interpolación cuadrática es una técnica iterativa utilizada para encontrar el mínimo de una función en un intervalo donde se supone que es unimodal. A diferencia de otros métodos más simples, este utiliza la información de tres puntos del intervalo para ajustar un polinomio de segundo grado (una parábola) que aproxima el comportamiento de la función.

En cada iteración, se construye una parábola que pasa por tres puntos conocidos de la función y se calcula el mínimo de dicha parábola. Este punto se toma como una nueva aproximación del mínimo real de la función. A continuación, se actualizan los puntos de trabajo, conservando aquellos que están más cerca del mínimo, y se repite el proceso.

```
function [xmin, fmin, iter, tiempo] = incu(f, a, b, tol)

% ---------------- INICIALIZACIÓN ----------------
tic; %Cronometro
iter = 0;
maxIter = 1000;


% Tres puntos iniciales
x1 = a;
x3 = b;
x2 = (a + b)/2;

f1 = f(x1);
f2 = f(x2);
f3 = f(x3);

% ---------------- BUCLE PRINCIPAL ----------------
while (x3 - x1) > tol && iter < maxIter

    % Ajuste cuadrático (resolver sistema)
    X = [x1^2, x1, 1;
         x2^2, x2, 1;
         x3^2, x3, 1];

    Y = [f1; f2; f3];

    coeffs = X \ Y;  % [a2; a1; a0]

    % Mínimo de la parábola
    x_min = -coeffs(2)/(2*coeffs(1));

    % Evaluar en nuevo punto
    f_min = f(x_min);

    % ---------------- ACTUALIZACIÓN ----------------
    % Mantener 3 puntos alrededor del mínimo

    if x_min < x2
        if f_min < f2
            x3 = x2; f3 = f2;
            x2 = x_min; f2 = f_min;
```

```
        else
            x1 = x_min; f1 = f_min;
        end
    else
        if f_min < f2
            x1 = x2; f1 = f2;
            x2 = x_min; f2 = f_min;
        else
            x3 = x_min; f3 = f_min;
        end
    end

    iter = iter + 1;
end

% ---------------- RESULTADO ----------------
xmin = x2;
fmin = f2;
tiempo = toc;

end
```

Método de Newton

El método de Newton es una técnica iterativa que permite encontrar el mínimo de una función utilizando información de sus derivadas. Se basa en la idea de que un mínimo local se encuentra en un punto donde la derivada de la función es cero. Para aproximar este punto, el método utiliza tanto la derivada primera como la segunda, lo que le permite tener en cuenta la curvatura de la función.

En cada iteración, se calcula una nueva aproximación del mínimo a partir de la anterior, aplicando una fórmula que depende de las derivadas. Este proceso se repite hasta que la diferencia entre iteraciones es suficientemente pequeña.

En esta práctica, el método de Newton no se ha aplicado a la función numérica (función 4), ya que no se dispone de una expresión analítica de sus derivadas. Dado que esta función se evalúa mediante la resolución de una ecuación diferencial, no es posible calcular de forma directa ni la derivada primera ni la segunda, lo que impide la aplicación del método.

```
function [xmin, fmin, iter, tiempo] = new(f, df, ddf, x0, tol)

% ---------------- INICIALIZACIÓN ----------------
tic; %Cronometro
iter = 0;
maxIter = 1000;
x = x0;

% ---------------- BUCLE PRINCIPAL ----------------
while iter < maxIter

    g = df(x);      % derivada
    H = ddf(x);     % segunda derivada
```

```
    % Evitar división por cero
    if abs(H) < 1e-12
        warning('Segunda derivada cercana a cero. Se detiene el
método.');
        break;
    end

    % Paso de Newton
    x_new = x - g/H;

    % Criterio de parada
    if abs(x_new - x) < tol
        break;
    end

    x = x_new;
    iter = iter + 1;
end

% ---------------- RESULTADO ----------------
xmin = x;
fmin = f(x);
tiempo = toc;

end
```

Método de las rectas inexactas

El método de rectas inexactas es un procedimiento iterativo que permite encontrar el mínimo de una función en un intervalo donde se supone que es unimodal, sin necesidad de calcular derivadas. El método se basa en evaluar la función en dos puntos interiores del intervalo y comparar sus valores para determinar en qué parte del intervalo se encuentra el mínimo.

En cada iteración, se seleccionan dos puntos dentro del intervalo y se evalúa la función en ellos. A partir de esta comparación, se descarta una parte del intervalo y se conserva la región donde es más probable que se encuentre el mínimo. Este proceso se repite hasta que el intervalo es suficientemente pequeño.

```
function [xmin, fmin, iter, tiempo] = rein(f, a, b, tol)

% ---------------- INICIALIZACIÓN ----------------
tic; %Cronometro
iter = 0;
maxIter = 1000;

% ---------------- BUCLE PRINCIPAL ----------------
while (b - a) > tol && iter < maxIter

    % Puntos internos (división simple)
    x1 = a + (b - a)/3;
    x2 = b - (b - a)/3;

    f1 = f(x1);
```

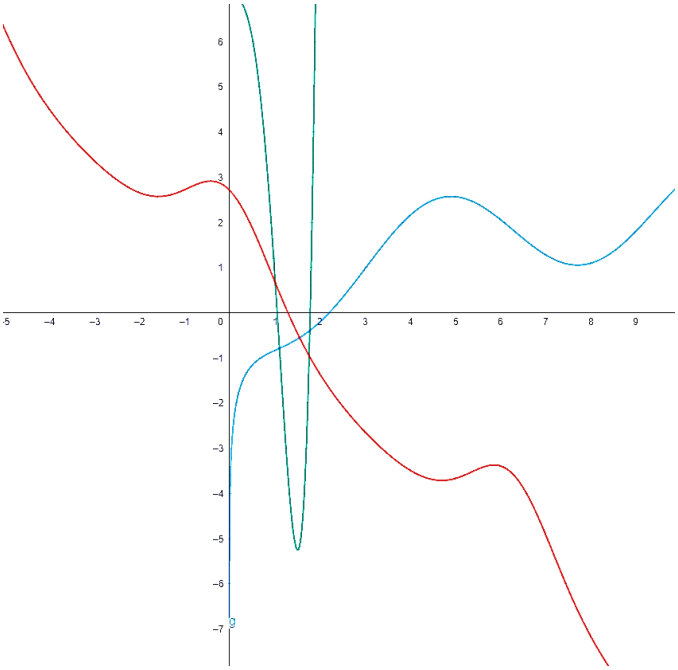
```
    f2 = f(x2);

    % Actualización del intervalo
    if f1 < f2
        b = x2;
    else
        a = x1;
    end

    iter = iter + 1;
end

% ---------------- RESULTADO ----------------
xmin = (a + b)/2;
fmin = f(xmin);
tiempo = toc;

end
```

Script principal de ejecución

A continuación, se presenta el código inicial para la ejecución de los algoritmos. En primer lugar, es necesario la declaración de las funciones y búsqueda de los intervalos unimodales de longitud 1. En el caso de las primera 3 funciones, se ha decidido implementarlas en GeoGebra debido a la sencillez de su definición, permitiendo así una rápida interpretación y obtención de un intervalo unimodal.

```
% ---------------- PARÁMETROS GENERALES ----------------

tol = 1e-5;
delta = 1e-4;

% ---------------- DEFINICIÓN DE FUNCIONES ----------------

f1 = @(x) x.^6 - 7*x.^3 + 7;
f2 = @(x) log(x) - sin(x);
f3 = @(x) exp(cos(x)) - x;
f4 = @funcionrr;

funciones = {f1, f2, f3, f4};
nombres = {'f1','f2','f3','funcionrr'};
n = length(funciones);
```

- $f(x) = x^6 - 7\,x^3 + 7$
- $g(x) = \ln(x) - \mathrm{sen}(x)$
- $h(x) = e^{\cos(x)} - x$



Viendo las graficas de cada función, se puede obtener los siguientes intervalos para cada función:

$$f1 \rightarrow [1,2]$$

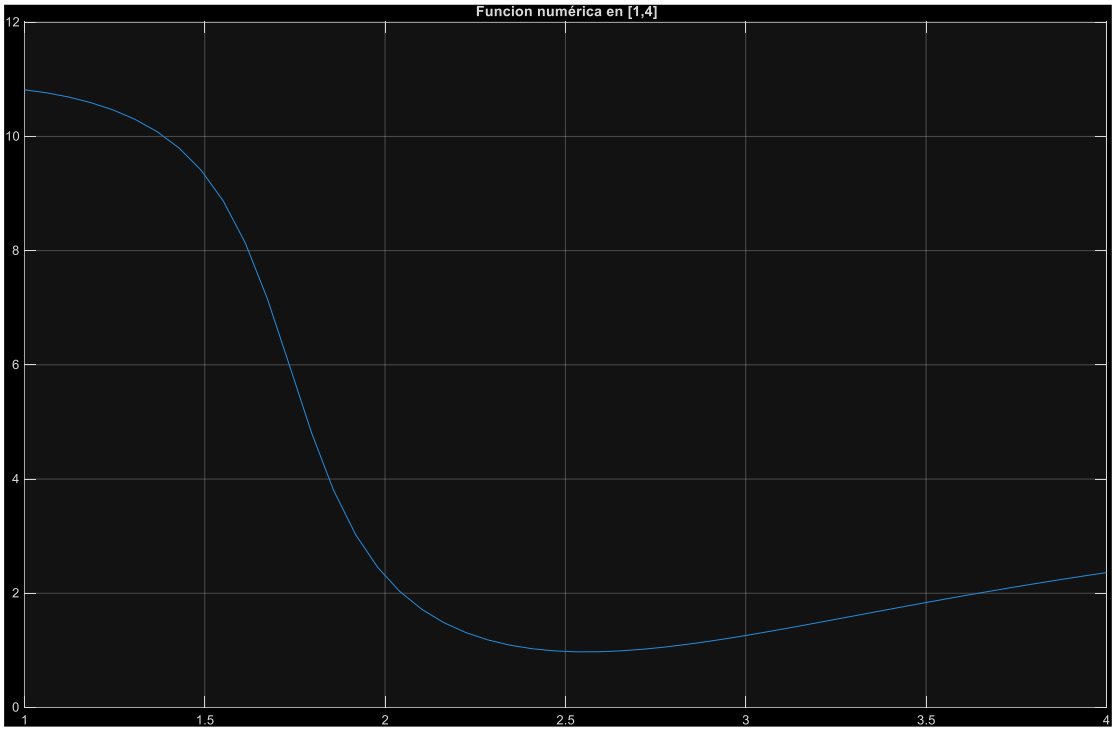
$$f2 \rightarrow [7,8]$$

$$f3 \rightarrow [4,5]$$

Todos estos intervalos son unimodales de longitud unitaria.

Para la cuarta función, ha sido necesario dibujarla utilizando Matlab ya que esta definida de forma numérica. Siguiendo las indicaciones dadas, se ha buscado el intervalo dentro del intervalo [1,4].

```
% ------------ DIBUJO DE LA CUARTA FUNCIÓN -------------

x = linspace(1,4,50);
y = arrayfun(f4, x);

plot(x,y)
grid on
title('Función numérica en [1,4]')
```



Para la última función, podemos apreciar que el intervalo necesario es:

$$f4 \rightarrow [2,3]$$

Teniendo ya estos cuatro intervalos encontrados, podemos utilizar las diferentes funciones para cada método y obtener el resultado. El siguiente bloque de código se encarga de inicializar la matriz de intervalos y las matrices de resultados para cada método.

```
% ---------------- INTERVALOS ----------------
intervalos = [
    1   2;
    7   8;
    4   5;
    2   3
];

% ---------------- RESULTADOS ----------------
res_budi = zeros(n,4);
res_incu = zeros(n,4);
res_new  = zeros(n,4);
res_rein = zeros(n,4);
```

Resultados de los métodos de búsqueda

En este apartado se presentan los resultados obtenidos de cada método. Esto nos permite comparar los resultados y seleccionar mejores o peores métodos, incluso diferenciando por función de ser lo correcto

Método de búsqueda dicotómica

El código necesario para ejecutar la búsqueda dicotómica es el siguiente:

```
for i = 1:n

    f = funciones{i};
    a = intervalos(i,1);
    b = intervalos(i,2);

    [xmin, fmin, iter, tiempo] = budi(f, a, b, tol, delta);

    res_budi(i,:) = [xmin, fmin, iter, tiempo];

    fprintf('Función: %s\n', nombres{i});
    fprintf('Intervalo: [%.2f, %.2f]\n', a, b);
    fprintf('xmin = %.6f\n', xmin);
    fprintf('f(xmin) = %.6f\n', fmin);
    fprintf('Iteraciones = %d\n', iter);
    fprintf('Tiempo = %.6f s\n\n', tiempo);
end
```

Este código permite además almacenar los resultados en una matriz que facilita la posterior visualización de los datos.

```
tabla_budi = array2table(res_budi, ...
    'VariableNames', {'xmin','fmin','iteraciones','tiempo'}, ...
    'RowNames', nombres);

disp('--- TABLA RESULTADOS BÚSQUEDA DICOTÓMICA ---');
disp(tabla_budi);
```

Por último, se muestran los datos obtenidos con la última ejecución del script para este método:

Función	xmin	fmin	iteraciones	tiempo
$f(x) = x^6 - 7x^3 + 7$	1.5183	-5.2500	1000	0.0017102
$f(x) = \ln x - \operatorname{sen} x$	7.7242	1.0528	1000	0.0012651
$f(x) = e^{\cos x} - x$	4.7124	-3.7124	1000	0.0007112
Funcionrr	2.5574	0.97258	1000	10.0010

Método de interpolación cuadrática

El código necesario para ejecutar la interpolación cuadrática es el siguiente:

```
for i = 1:n
    f = funciones{i};
    a = intervalos(i,1);
    b = intervalos(i,2);

    [xmin, fmin, iter, tiempo] = incu(f, a, b, tol);

    res_incu(i,:) = [xmin, fmin, iter, tiempo];

    fprintf('Función: %s\n', nombres{i});
    fprintf('Intervalo: [%.2f, %.2f]\n', a, b);
    fprintf('xmin = %.6f\n', xmin);
    fprintf('f(xmin) = %.6f\n', fmin);
```

```
    fprintf('Iteraciones = %d\n', iter);
    fprintf('Tiempo = %.6f s\n\n', tiempo);
end
```

Este código permite además almacenar los resultados en una matriz que facilita la posterior visualización de los datos.

```
tabla_incu = array2table(res_incu, ...
    'VariableNames', {'xmin','fmin','iteraciones','tiempo'}, ...
    'RowNames', nombres);

disp('--- TABLA RESULTADOS INTERPOLACIÓN CUADRÁTICA ---');
disp(tabla_incu);
```

Por último, se muestran los datos obtenidos con la última ejecución del script para este método:

Función	xmin	fmin	iteraciones	tiempo
$f(x) = x^6 - 7x^3 + 7$	1.5183	-5.2500	31	0.0068288
$f(x) = \ln x - \operatorname{sen} x$	7.7242	1.0528	7	0.0010229
$f(x) = e^{\cos x} - x$	4.7124	-3.7124	8	0.0071775
Funcionrr	2.5574	0.97258	11	0.0732860

Método de Newton

El código necesario para ejecutar el método de Newton es el siguiente:

```
df1 = @(x) 6*x.^5 - 21*x.^2;
ddf1= @(x) 30*x.^4 - 42*x;

df2 = @(x) 1./x - cos(x);
ddf2= @(x) -1./x.^2 + sin(x);

df3 = @(x) -exp(cos(x)).*sin(x) - 1;
ddf3= @(x) exp(cos(x)).*(sin(x).^2 - cos(x));

derivadas = {df1, df2, df3};
segundas  = {ddf1, ddf2, ddf3};

for i = 1:n

    % No aplicar a funcionrr
    if i == 4
        res_new(i,:) = [NaN, NaN, NaN, NaN];
        continue;
    end

    f = funciones{i};
    df  = derivadas{i};
    ddf = segundas{i};

    a = intervalos(i,1);
    b = intervalos(i,2);

    x0 = (a + b)/2; % Punto inicial

    [xmin, fmin, iter, tiempo] = new(f, df, ddf, x0, tol);
```

```
    res_new(i,:) = [xmin, fmin, iter, tiempo];

    fprintf('Función: %s\n', nombres{i});
    fprintf('Intervalo: [%.2f, %.2f]\n', a, b);
    fprintf('xmin = %.6f\n', xmin);
    fprintf('f(xmin) = %.6f\n', fmin);
    fprintf('Iteraciones = %d\n', iter);
    fprintf('Tiempo = %.6f s\n\n', tiempo);
end
```

Este código permite además almacenar los resultados en una matriz que facilita la posterior visualización de los datos.

```
tabla_new = array2table(res_new, ...
    'VariableNames', {'xmin','fmin','iteraciones','tiempo'}, ...
    'RowNames', nombres);

disp('--- TABLA RESULTADOS MÉTODO DE NEWTON ---');
disp(tabla_new);
```

Por último, se muestran los datos obtenidos con la última ejecución del script para este método:

Función	xmin	fmin	iteraciones	tiempo
$f(x) = x^6 - 7x^3 + 7$	1.5183	-5.2500	2	0.0004397
$f(x) = \ln x - \operatorname{sen} x$	7.7242	1.0528	2	0.0010026
$f(x) = e^{\cos x} - x$	4.7124	-3.7124	2	0.0012786
Funcionrr	NaN	NaN	NaN	NaN

Método de las rectas inexactas

El código necesario para ejecutar el método de las rectas inexactas es el siguiente:

```
for i = 1:n
    f = funciones{i};
    a = intervalos(i,1);
    b = intervalos(i,2);

    [xmin, fmin, iter, tiempo] = rein(f, a, b, tol);

    res_rein(i,:) = [xmin, fmin, iter, tiempo];

    fprintf('Función: %s\n', nombres{i});
    fprintf('Intervalo: [%.2f, %.2f]\n', a, b);
    fprintf('xmin = %.6f\n', xmin);
    fprintf('f(xmin) = %.6f\n', fmin);
    fprintf('Iteraciones = %d\n', iter);
    fprintf('Tiempo = %.6f s\n\n', tiempo);
end
```

Este código permite además almacenar los resultados en una matriz que facilita la posterior visualización de los datos.

```
tabla_rein = array2table(res_rein, ...
    'VariableNames', {'xmin','fmin','iteraciones','tiempo'}, ...
    'RowNames', nombres);

disp('--- TABLA RESULTADOS RECTAS INEXACTAS ---');
```

```
disp(tabla_rein);
```

Por último, se muestran los datos obtenidos con la última ejecución del script para este método:

Función	xmin	fmin	iteraciones	tiempo
$f(x) = x^6 - 7x^3 + 7$	1.5183	-5.2500	29	0.0008598
$f(x) = \ln x - \operatorname{sen} x$	7.7242	1.0528	29	0.0007455
$f(x) = e^{\cos x} - x$	4.7124	-3.7124	29	0.0002986
Funcionrr	2.5574	0.97258	29	0.3023900

Análisis de los resultados

El primer dato que destaca entre todos los métodos es la cantidad de iteraciones. En el caso de la búsqueda dicotómica se han tenido que realizar 1000 iteraciones. Se debe tener en cuenta que se impuso un limite de iteraciones para evitar posibles fallos o infinitas iteraciones. Esto implica que la convergencia hacia el mínimo es lenta. Se debe a que con cada iteración se reduce solo el intervalo de búsqueda sin utilizar información adicional sobre la función. Además, el tiempo es elevado para la función numérica debido al coste de ejecución de la función con cada evaluación.

El método de interpolación cuadrática mejora el número de iteraciones de forma significativa, lo que supone una convergencia mas rápida. Se debe a la aproximación parabólica, que permite estimar el mínimo de forma mas precisa con cada iteración. El problema surge con los tiempos de ejecución, que cada iteración supone cálculos adicionales.

El método de Newton es el más eficiente en términos de numero de iteraciones. Se debe a que aprovecha la información de la derivada y la segunda derivada para acercarse al mínimo. Este método requiere una elección buena del punto inicial y además no es aplicable a funciones que no tengan una expresión analítica de sus derivadas, como es el caso de la función numérica. Para realizar este método en funciones numéricas, es necesario realizar el cálculo de la derivada de la función de forma numérica con la definición de derivada $\lim_{h \to o} \frac{f(x+h)-f(x-h)}{2*h}$. Al realizar programar la función de primera derivada y segunda derivada para la función numérica, se podría finalmente ejecutar el método de newton para esta función. Esta solución tiene un problema, el coste de ejecución y el posible error que puede llegar a dar. Con cada evaluación de la derivada se debe resolver una EDO dos veces y con la segunda derivada serian tres veces. Aun así, se presenta la propuesta para realizar el método, los datos obtenidos donde se puede observar el coste que supone y la modificación a los códigos para poder implementarlo:

Función	xmin	fmin	iteraciones	tiempo
Funcionrr	2.5574	0.97258	3	0.10176

Ha sido necesario añadir código al script principal

```
f = funciones{4};

% Primera derivada
```

```
df = @(f, x, h) (f(x + h) - f(x - h)) / (2*h);

% Segunda derivada. Se ha evitado la opción de derivar la derivada para
% reducir los costes de su ejecución.
ddf = @(f, x, h) (f(x + h) - 2*f(x) + f(x - h)) / (h^2);

a = intervalos(4,1);
b = intervalos(4,2);
x0 = (a + b)/2; % Punto inicial

[xmin, fmin, iter, tiempo] = new_numerical(f, df, ddf, x0, tol);
res_new(4,:) = [xmin, fmin, iter, tiempo];

fprintf('Función: %s\n', nombres{i});
fprintf('Intervalo: [%.2f, %.2f]\n', a, b);
fprintf('xmin = %.6f\n', xmin);
fprintf('f(xmin) = %.6f\n', fmin);
fprintf('Iteraciones = %d\n', iter);
fprintf('Tiempo = %.6f s\n\n', tiempo);
```

Ha sido necesario crear una copia del método de Newton modificada para poder realizarse sobre funciones numéricas y que se realicen cálculos numéricos de las derivadas:

```
g = df(@funcionrr, x, h);      % derivada
H = ddf(@funcionrr, x, h);     % segunda derivad
```

Además, se ha inicializado la variable h = 1e-5 junto a las demás variables de inicio para poder realizar las derivadas.

Por último, el método de rectas inexactas presenta un comportamiento nada destacable. No supone muchas iteraciones, aunque si es mas lento que el método de Newton. Es un método adecuado para funciones numéricas ya que no supone el calculo de derivadas. Si el coste de evaluación de la función fuese realmente alto, el continuo cálculo de derivadas numéricas para realizar el método de Newton supondría un coste demasiado elevado, algo que este método podría solucionar.

En conclusión, el mejor método es el de Newton para funciones analíticas y funciones numéricas de bajo coste. Cuando sube el coste de evaluar las funciones analíticas, es necesario pasar a utilizar métodos como la interpolación cuadrática o las rectas inexactas. En cualquier caso, el método de búsqueda dicotómica es robusto pero supone una cantidad de iteraciones demasiado alta y un coste alto por lo que es el método menos eficiente de los vistos.

Enlace al repositorio de GitHub

Repositorio